

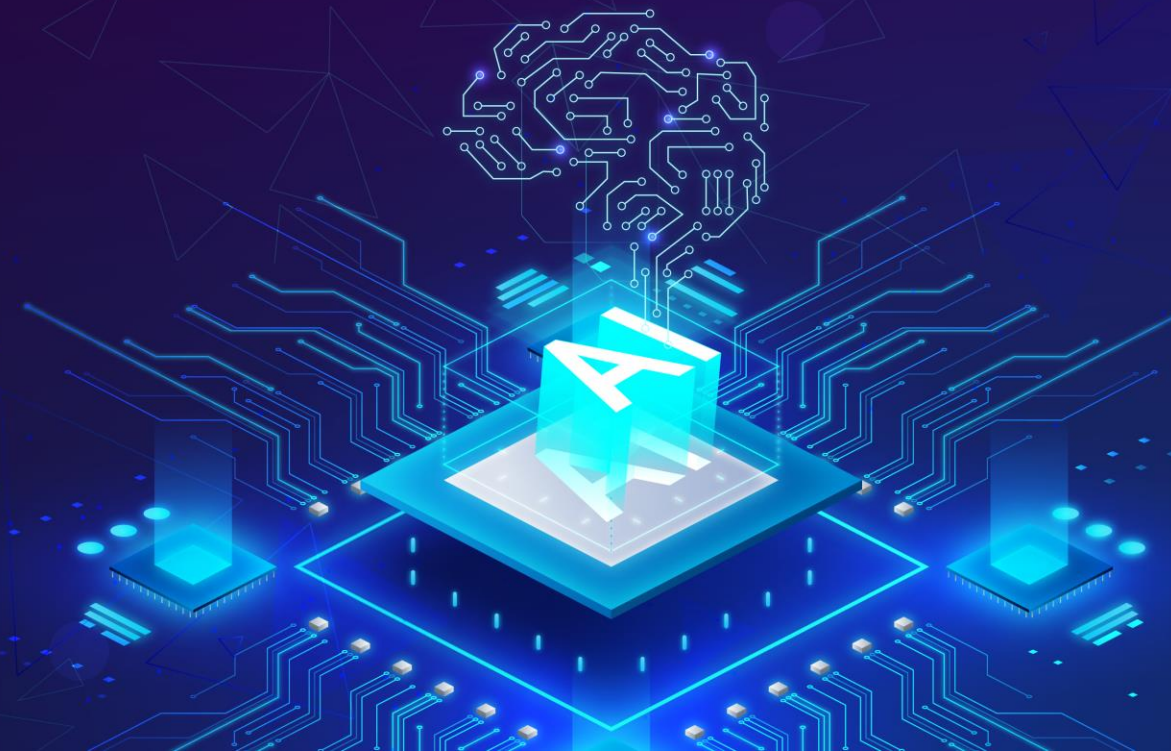


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Nhóm chuyên môn Trí tuệ nhân tạo

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Tìm kiếm DFS-DLS-IDS



- 1. Tìm kiếm DFS**
- 2. Tìm kiếm DLS, IDS**
- 3. So sánh các giải thuật tìm kiếm cơ bản**

Bài học này cung cấp cho người học:

1. Thuật toán tìm kiếm theo chiều sâu DFS.
2. Thuật toán tìm kiếm giới hạn độ sâu LDS, sâu dần IDS.
3. So sánh chỉ số thực thi của các giải thuật tìm kiếm cơ bản.

1. Tìm kiếm DFS

1.1. Thuật toán

1.2. Minh họa

1.3. Đánh giá

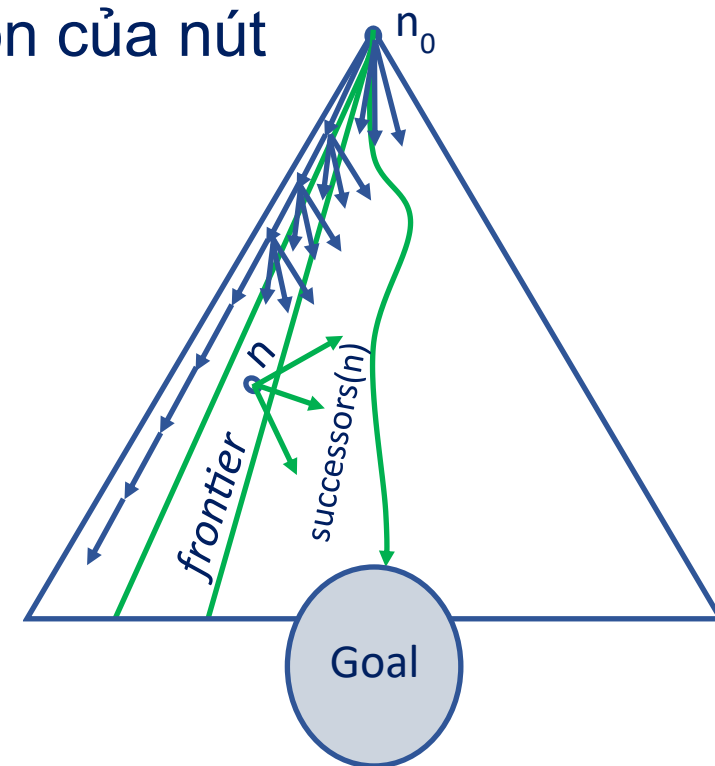
2. Tìm kiếm DLS, IDS

3. So sánh các giải thuật tìm kiếm cơ bản

1. TÌM KIẾM DFS

1.1. Thuật toán

- Tìm kiếm theo chiều sâu trước: mở rộng các nút sâu nhất trước
 - Mở rộng nút gốc, một nút con của nút gốc, một nút con của nút vừa mở rộng... đến nút lá
 - Tiếp tục mở rộng các nhánh khác của cây tìm kiếm
 - Mở rộng theo chiều dọc (VD từ trái sang phải)
- Cài đặt
 - **Frontier** là hàng đợi **LIFO** (ngăn xếp)
 - Các nút (trong hàng đợi) được xem xét theo thứ tự độ sâu giảm dần



Hình 1.1: Minh họa tìm kiếm DFS

1. TÌM KIẾM DFS

1.1. Thuật toán

- Phương pháp tìm kiếm theo cấu trúc cây

DFS(**root**, **goals**) returns **a solution node** or **failure**

frontier $\leftarrow$ [root]

while NOT-EMPTY(frontier) do

$n \leftarrow$ **POP**(frontier)

 if ($n \in$ goals) then return n

 frontier = **children**(n) + frontier

return failure

Cơ chế **LIFO** của *frontier*:

- Thêm vào đầu $\langle \text{frontier} = \text{children}(n) + \text{frontier} \rangle$
- Lấy ra ở đầu frontier $\langle n \leftarrow \text{POP}(\text{frontier}) \rangle$

1. TÌM KIẾM DFS

1.1. Thuật toán

- Phương pháp đệ quy

DFS(**root**, **goals**) returns **a solution node** or **failure**

 return DFS_REC(**root**, **goals**)

DFS_REC(**n**, **goals**)

 if (**n** ∈ **goals**) then return **n**

 for **child** in children(**n**) do

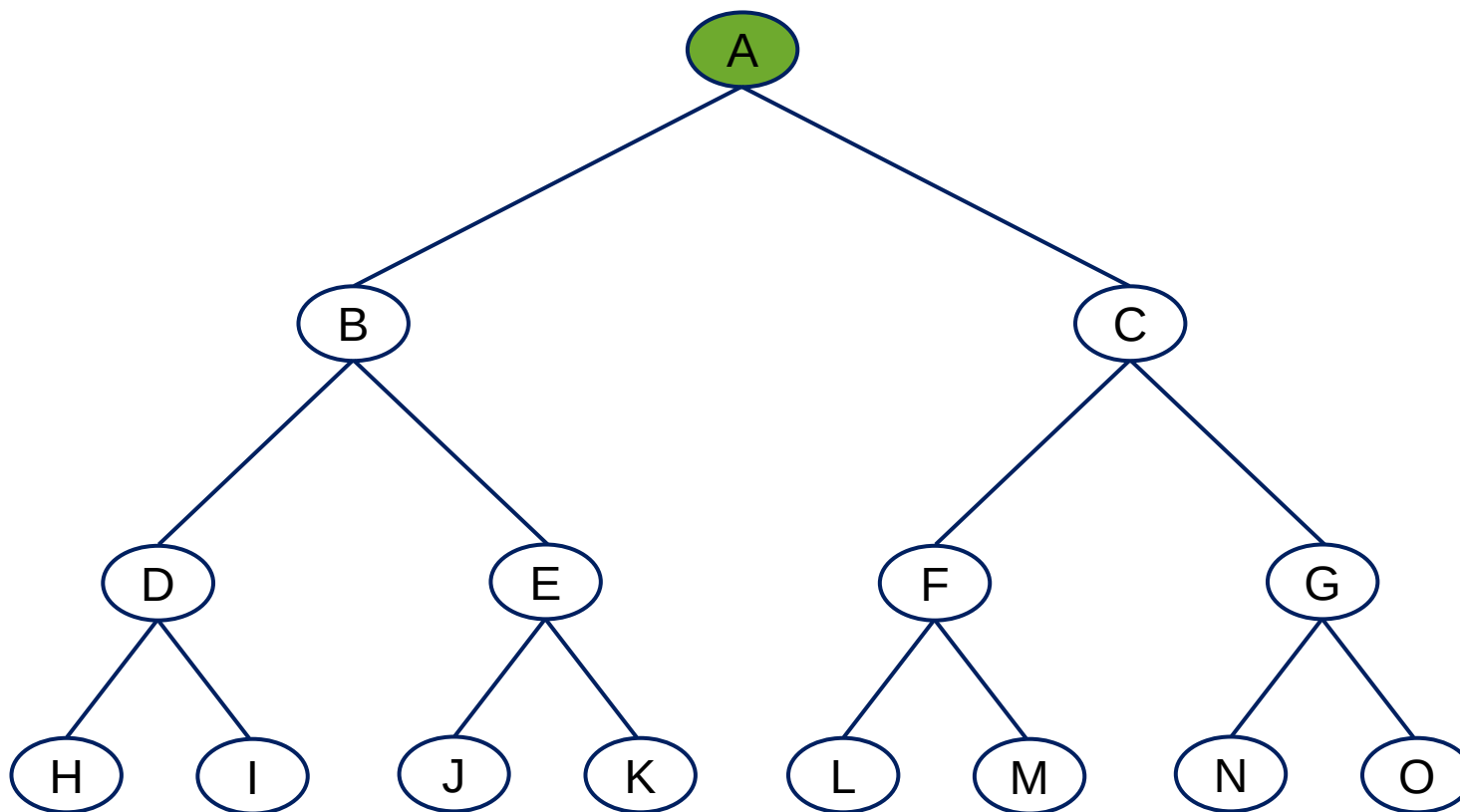
result = DFS_REC(**child**, **goals**)

 if **result** != failure then return **result**

 return failure

1. TÌM KIẾM DFS

1.2. Minh họa

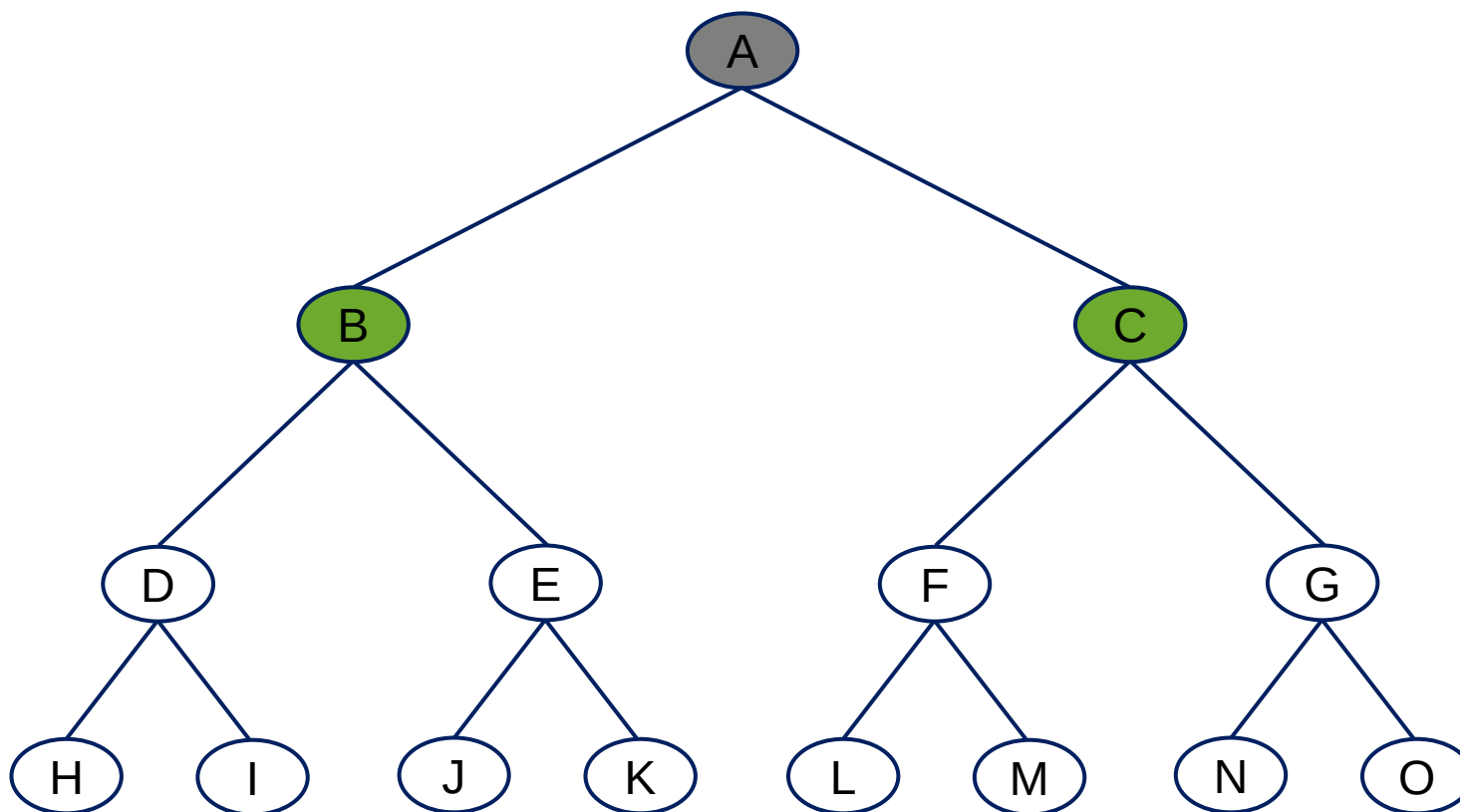


Frontier:
[A]

Hình 1.2: Minh họa thứ tự mở rộng cây tìm kiếm DFS

1. TÌM KIẾM DFS

1.2. Minh họa

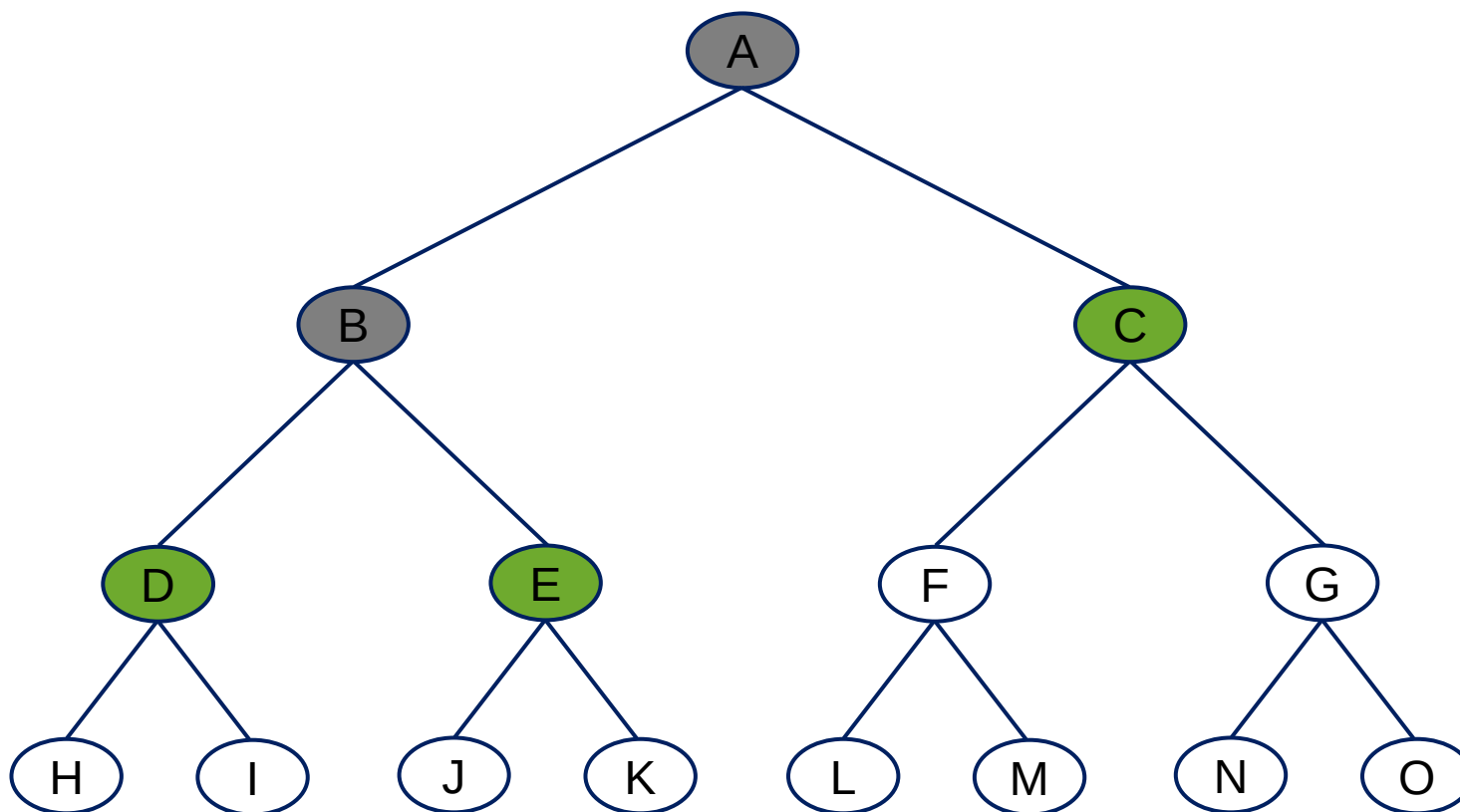


Frontier:
[A]
[B,C]

Hình 1.2: Minh họa thứ tự mở rộng cây tìm kiếm DFS

1. TÌM KIẾM DFS

1.2. Minh họa



Frontier:

[A]

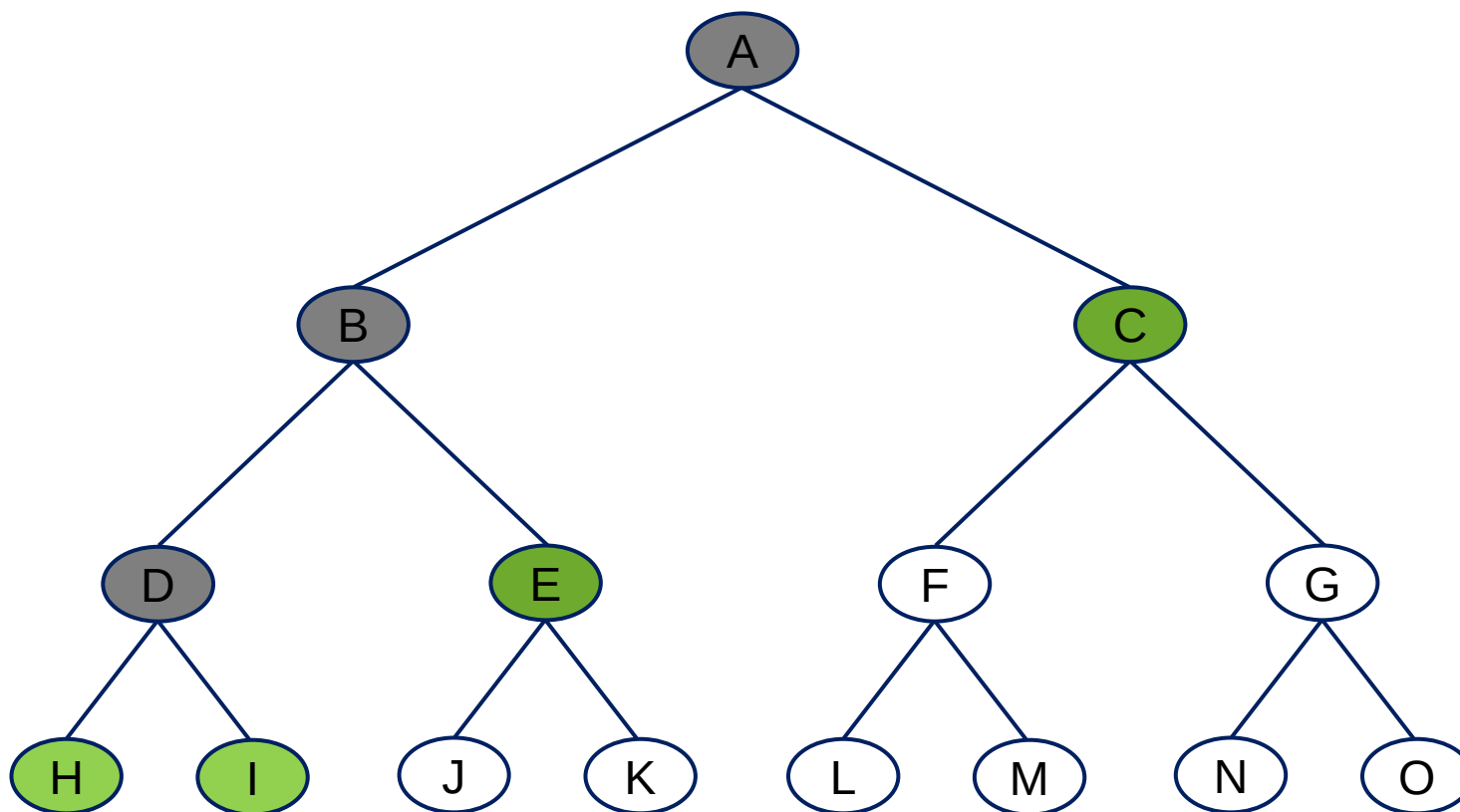
[B,C]

[D,E,C]

Hình 1.2: Minh họa thứ tự mở rộng cây tìm kiếm DFS

1. TÌM KIẾM DFS

1.2. Minh họa

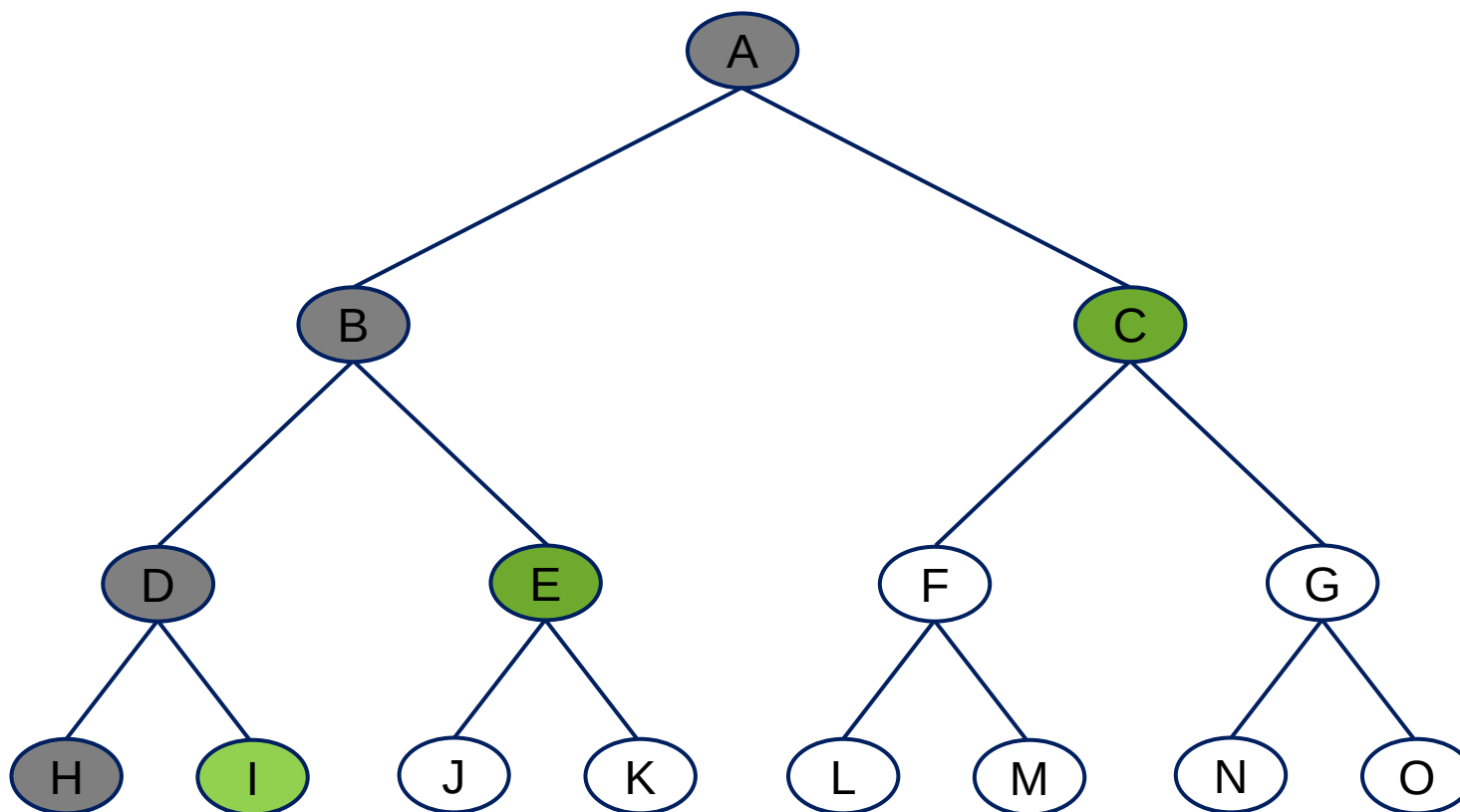


Frontier:
[A]
[B,C]
[D,E,C]
[H,I,E,C]

Hình 1.2: Minh họa thứ tự mở rộng cây tìm kiếm DFS

1. TÌM KIẾM DFS

1.2. Minh họa

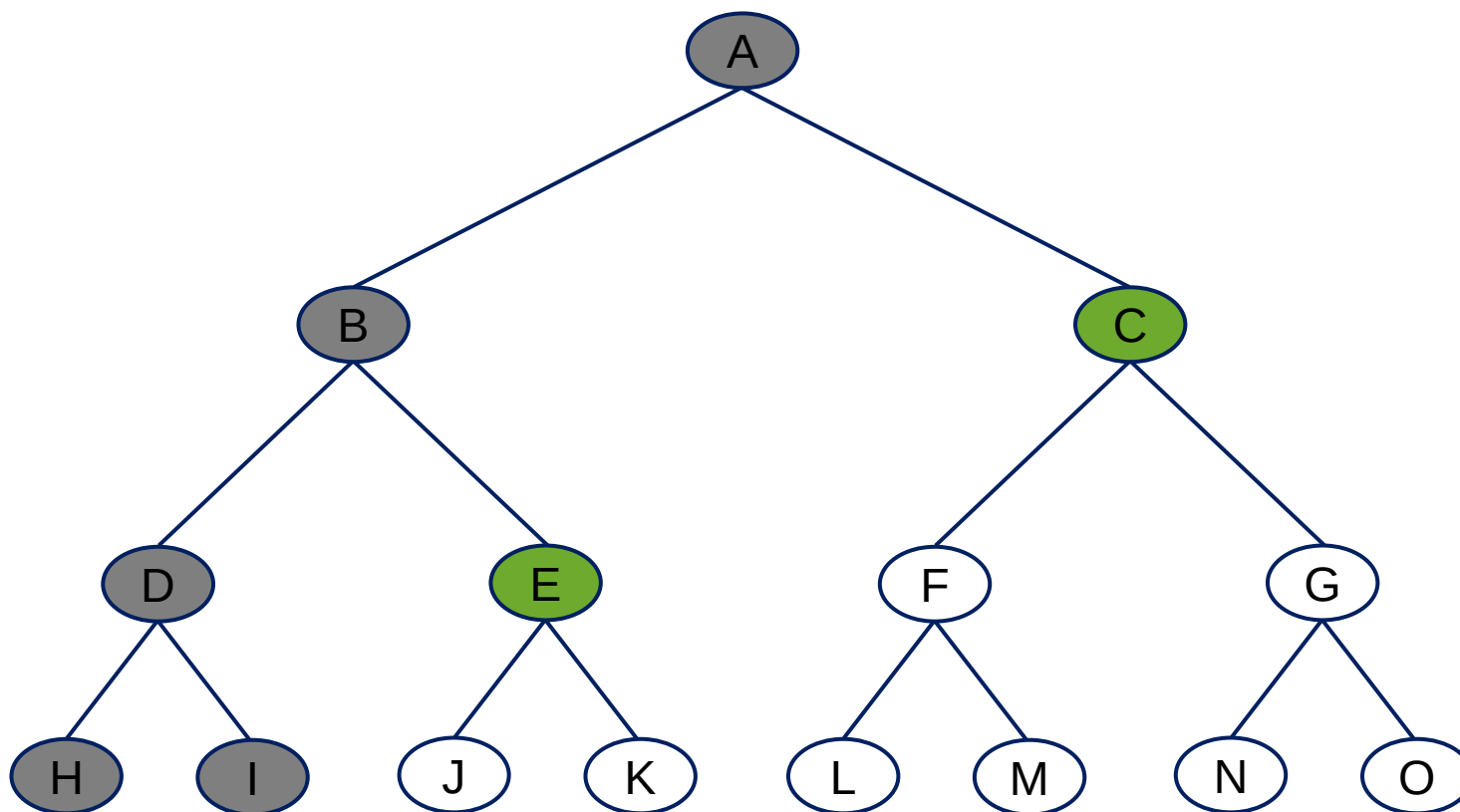


Frontier:
[A]
[B,C]
[D,E,C]
[H,I,E,C]
[I,E,C]

Hình 1.2: Minh họa thứ tự mở rộng cây tìm kiếm DFS

1. TÌM KIẾM DFS

1.2. Minh họa

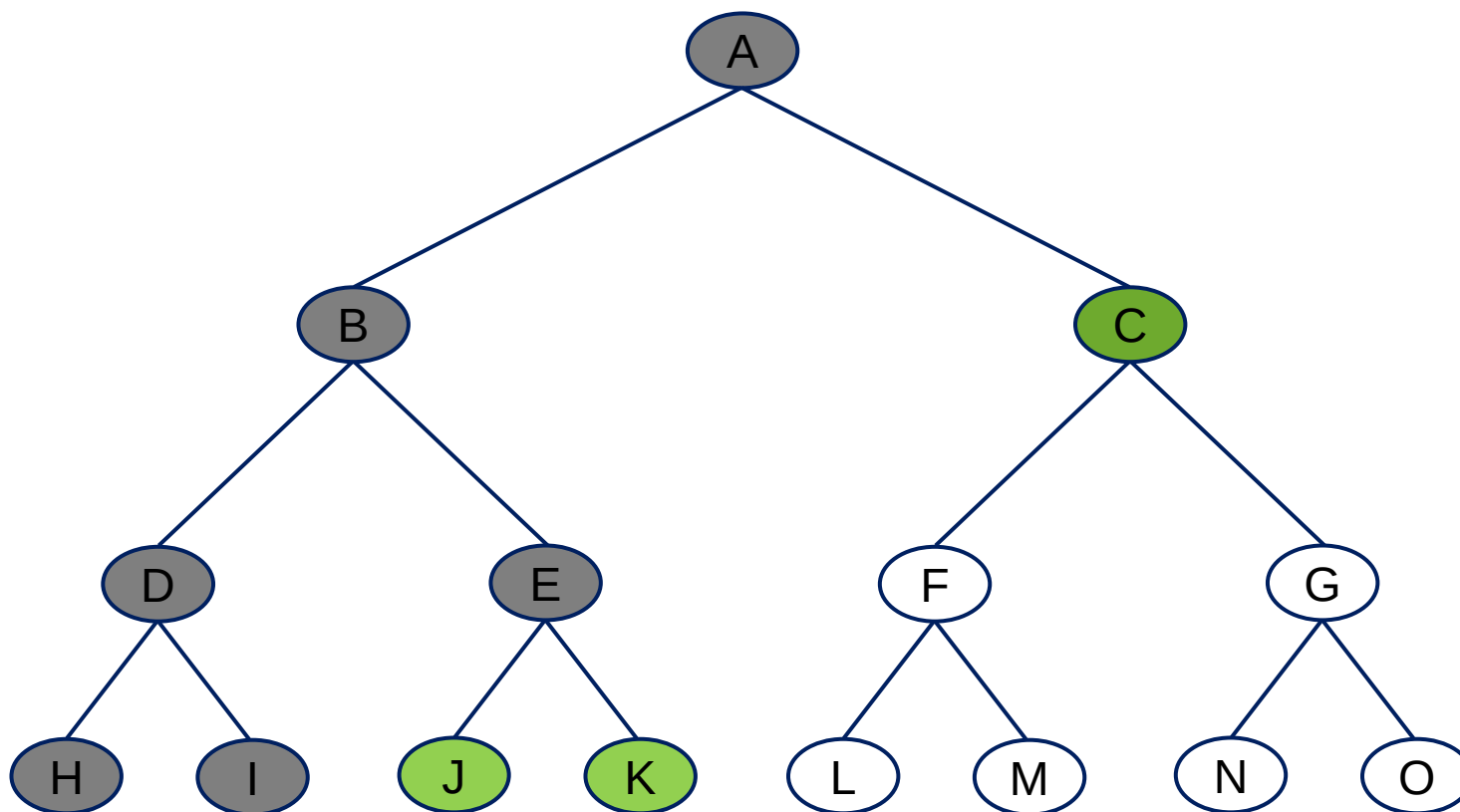


Frontier:
[A]
[B,C]
[D,E,C]
[H,I,E,C]
[I,E,C]
[E,C]

Hình 1.2: Minh họa thứ tự mở rộng cây tìm kiếm DFS

1. TÌM KIẾM DFS

1.2. Minh họa

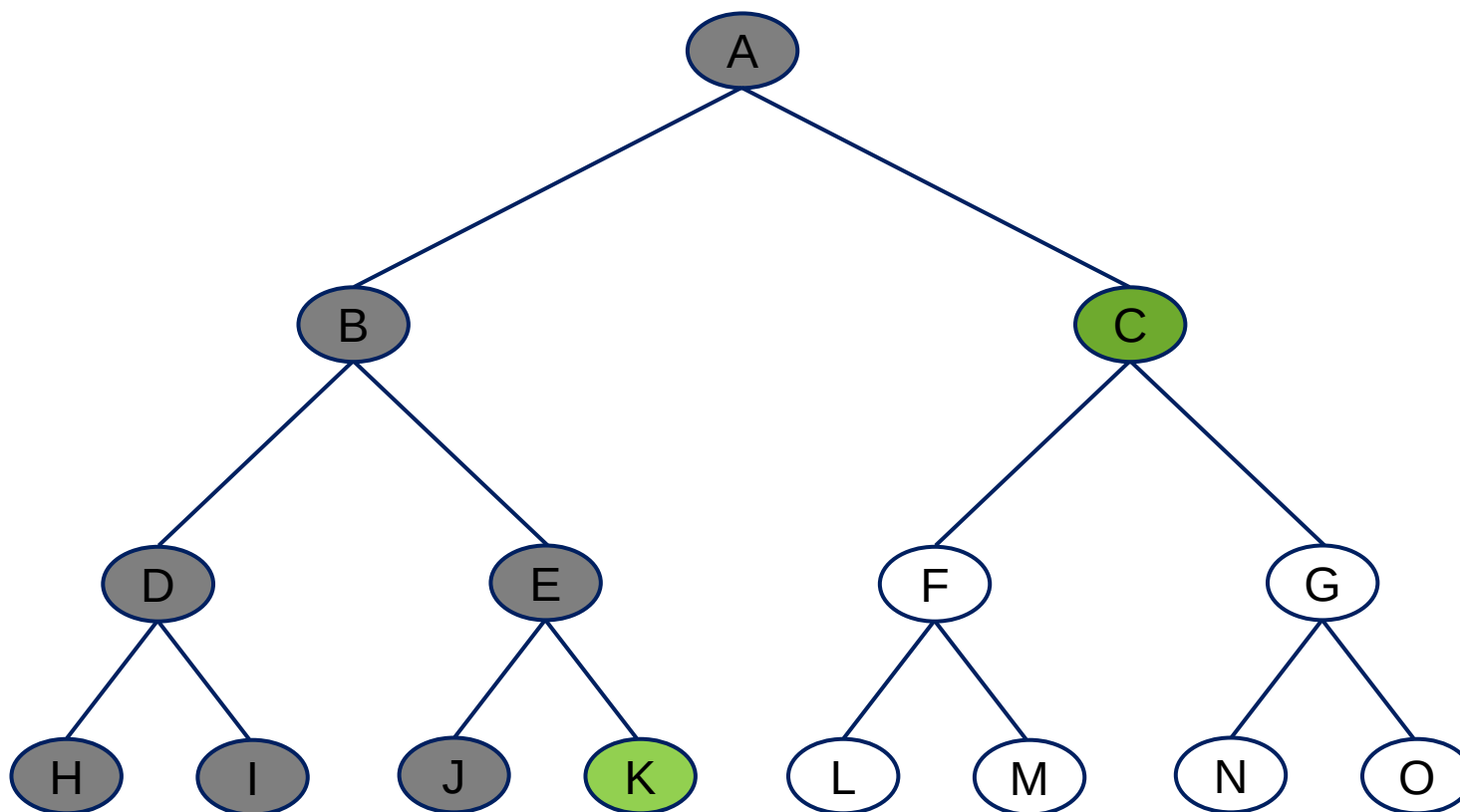


Frontier:
[A]
[B,C]
[D,E,C]
[H,I,E,C]
[I,E,C]
[E,C]
[J,K,C]

Hình 1.2: Minh họa thứ tự mở rộng cây tìm kiếm DFS

1. TÌM KIẾM DFS

1.2. Minh họa

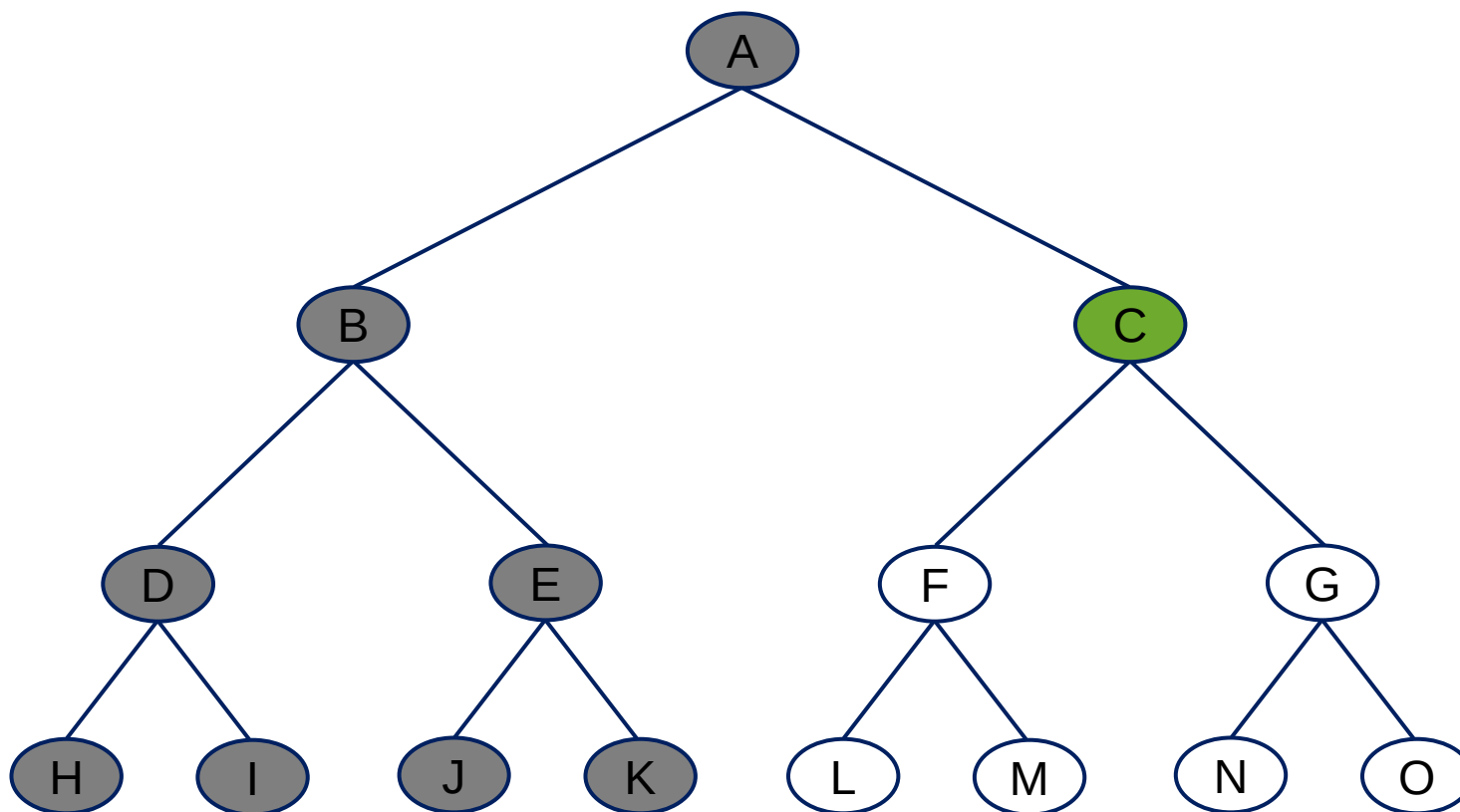


Frontier:
[A]
[B,C]
[D,E,C]
[H,I,E,C]
[I,E,C]
[E,C]
[J,K,C]
[K,C]

Hình 1.2: Minh họa thứ tự mở rộng cây tìm kiếm DFS

1. TÌM KIẾM DFS

1.2. Minh họa

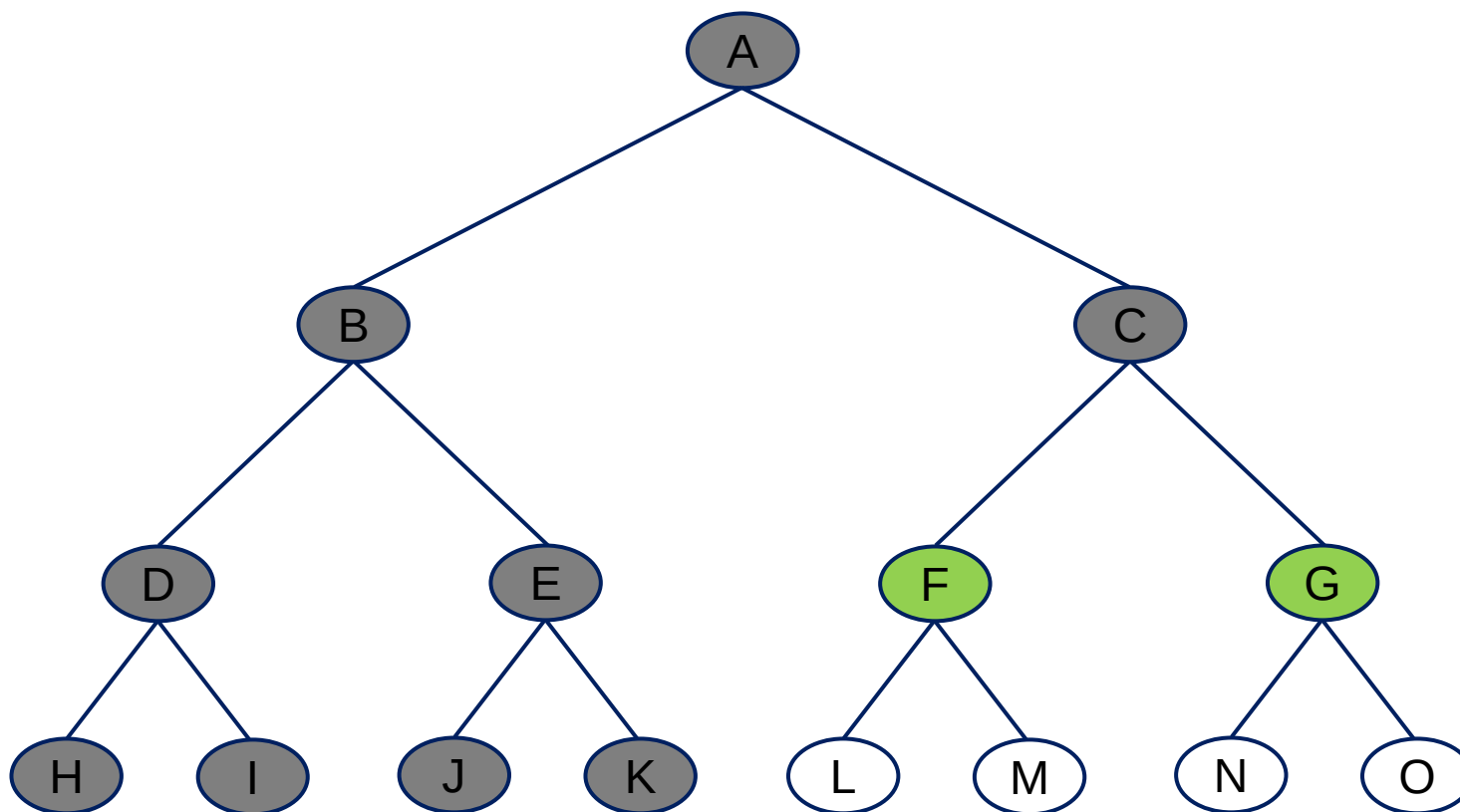


Frontier:
[A]
[B,C]
[D,E,C]
[H,I,E,C]
[I,E,C]
[E,C]
[J,K,C]
[K,C]
[C]

Hình 1.2: Minh họa thứ tự mở rộng cây tìm kiếm DFS

1. TÌM KIẾM DFS

1.2. Minh họa



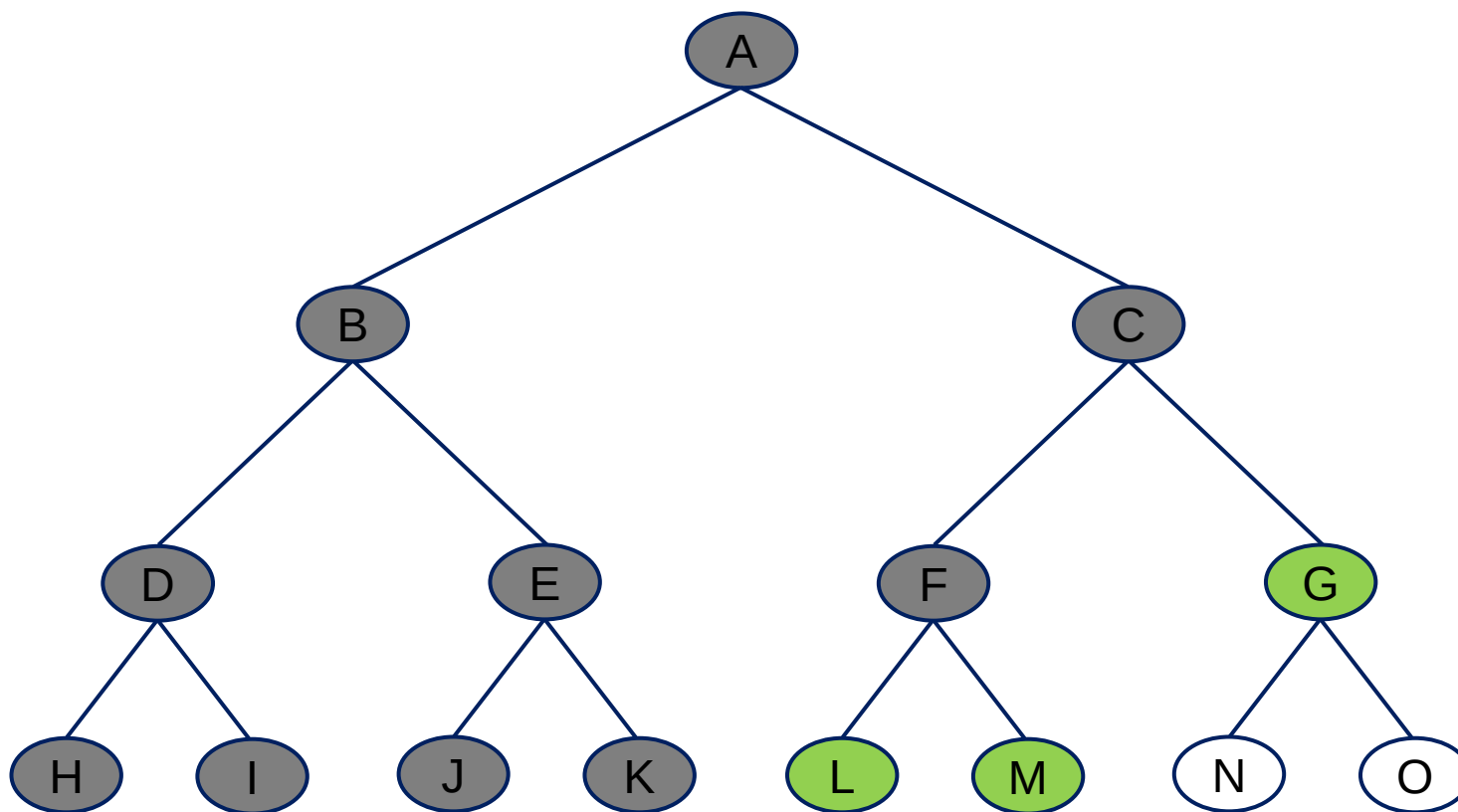
Frontier:

- [A]
- [B,C]
- [D,E,C]
- [H,I,E,C]
- [I,E,C]
- [E,C]
- [J,K,C]
- [K,C]
- [C]
- [F,G]**

Hình 1.2: Minh họa thứ tự mở rộng cây tìm kiếm DFS

1. TÌM KIẾM DFS

1.2. Minh họa



Frontier:

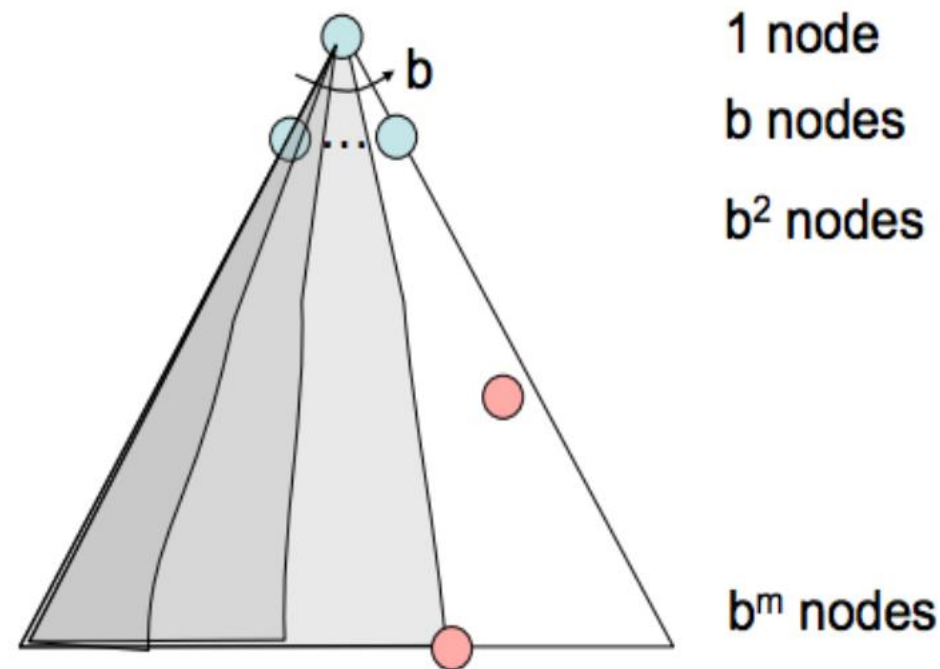
[A]
[B,C]
[D,E,C]
[H,I,E,C]
[I,E,C]
[E,C]
[J,K,C]
[K,C]
[C]
[F,G]
[L,M,G]

Hình 1.2: Minh họa thứ tự mở rộng cây tìm kiếm DFS

1. TÌM KIẾM DFS

1.3. Đánh giá

- Tính hoàn chỉnh?
 - **Không** (trường hợp không gian trạng thái có độ sâu vô hạn)
- Độ phức tạp về thời gian?
 - $O(b^m)$
- Độ phức tạp về bộ nhớ?
 - $O(bm)$
- Tính tối ưu?
 - **Không**



Hình 1.3: Minh họa đánh giá DFS

1. Tìm kiếm DFS

2. Tìm kiếm DLS, IDS

2.1. Thuật toán DLS

2.2. Thuật toán IDS

2.3. Đánh giá

3. So sánh các giải thuật tìm kiếm cơ bản

2. TÌM KIẾM DLS, IDS



2.1. Thuật toán DLS

- Tìm kiếm giới hạn độ sâu DLS (Depth limited search)
 - **DLS** = DFS + giới hạn độ sâu
 - Không xem xét, mở rộng các nút con của các nút độ sâu **depth_limit**

2. TÌM KIẾM DLS, IDS

2.1. Thuật toán DLS

DLS(**root**, **goals**, **depth_limit**) returns **a solution node** or **failure**

frontier $\leftarrow$ [root]

while NOT-EMPTY(frontier) do

$n \leftarrow$ POP(frontier)

 if ($n \in$ goals) then return n

If DEPTH(n) < **depth_limit**

 frontier = childrent(n , tree) + frontier

return failure

Sửa giải thuật DFS, chỉ mở rộng các nút con của các nút có độ sâu nhỏ hơn giới hạn (**depth_limit**)

2. TÌM KIẾM DLS, IDS



2.2. Thuật toán IDS

- Vấn đề DLS
 - Không xác định được độ sâu giới hạn (`depth_limit`) tối ưu
- Tìm kiếm sâu dần IDS (Iterative depth search)
 - Áp dụng DFS độ sâu giới hạn `depth_limit = 0`
 - Nếu thất bại, áp dụng DFS độ sâu giới hạn `depth_limit = 1`
 - Tiếp tục **tăng `depth_limit`** đến khi tìm được lời giải.

2. TÌM KIẾM DLS, IDS

2.2. Thuật toán IDS

IDS(**root**, **goals**) returns **a solution node** or **failure**

for depth_limit = 0 to ∞ do

result = DLS(**root**, **goals**, depth_limit)

If **result** != failure

return **result**

Trường hợp không
gian trạng thái có
độ sâu hữu hạn (m),
thay thế $\infty \leftarrow m$

2. TÌM KIẾM DLS, IDS



2.3. Đánh giá

- **DLS**: chỉ khác với DFS dừng ở độ sâu $l = \text{depth_limit}$
 - Tính hoàn chỉnh, tối ưu: **không**
 - Độ phức tạp về thời gian: $O(b^m) \rightarrow O(b^l)$
 - Độ phức tạp về bộ nhớ: $O(bm) \rightarrow O(bl)$
- **IDS**
 - Tính hoàn chỉnh, tối ưu: **tương tự BFS** (tìm các lời giải ở độ sâu 0, 1, 2...)
 - Độ phức tạp về thời gian: (chi tiết tại slide sau)
 - Độ phức tạp về bộ nhớ: $O(bm) \rightarrow O(bd)$ (d là độ sâu của lời giải)

2. TÌM KIẾM DLS, IDS



2.3. Đánh giá

- Độ phức tạp thời gian IDS
 - Số lượng các nút xét với DLS (độ sâu d và hệ số phân nhánh b):
 - $N_{DLS} = b^0 + b^1 + b^2 + \dots + b^{d-1} + b^d$
 - Số lượng các nút được sinh ra trong giải thuật tìm kiếm sâu dần là:
 - $N_{IDS} = (d+1)*b^0 + d*b^1 + (d-1)*b^2 + \dots + 2*b^{d-1} + 1*b^d$
 - Độ phức tạp thời gian cùng là $O(b^d)$.
 - VD $b = 10$, $d = 5$, chênh lệch là 11%

1. Tìm kiếm DFS

2. Tìm kiếm DLS, IDS

3. So sánh các giải thuật tìm kiếm cơ bản

3. SO SÁNH CÁC THUẬT TOÁN TÌM KIẾM CƠ BẢN

Tiêu chí	BFS	UCS	DFS	DLS	IDS
Tính hoàn chỉnh	Yes	Yes	No	No	Yes
Độ phức tạp thời gian	$O(b^d)$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Độ phức tạp bộ nhớ	$O(b^d)$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$
Tính tối ưu	Yes*	Yes	No	No	Yes*

1. Bài học đã cung cấp cho người học ý tưởng, ví dụ, giải thuật và đánh giá phương pháp **tìm kiếm theo chiều sâu DFS**.
2. Bài học trình bày ý tưởng, giải thuật và đánh giá phương pháp tìm kiếm với **chiều sâu giới hạn DLS, sâu dần IDS**.
3. Bài học cũng tổng hợp và so sánh chỉ số thực thi của các giải thuật **tìm kiếm cơ bản** đã học.

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

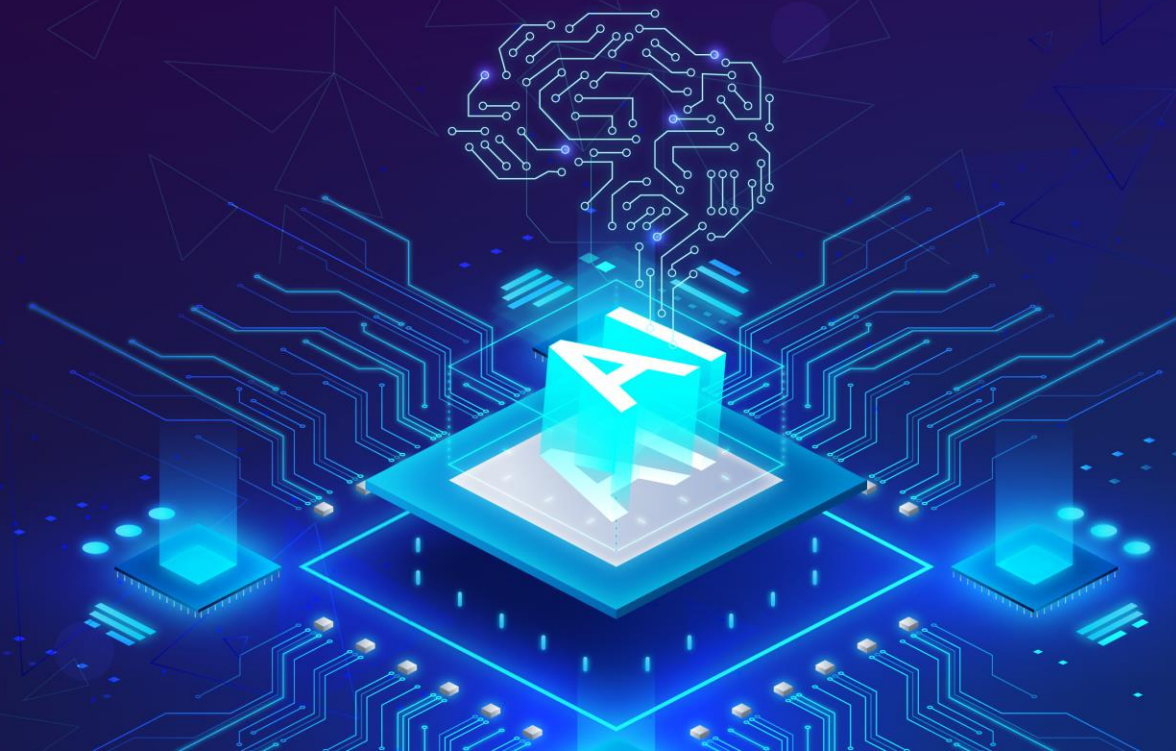
Tìm kiếm DFS-DLS-IDS

Biên soạn:

TS. Đỗ Tiến Dũng

Trình bày:

TS. Đỗ Tiến Dũng



NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Bài học tiếp theo:

Tìm kiếm tốt nhất trước

Tài liệu tham khảo:

- [1] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach* (4th Edition). Prentice Hall, 2020
- [2] Từ Minh Phương. *Giáo trình nhập môn trí tuệ nhân tạo*. NXB Thông tin và truyền thông, 2016
- [3] T. M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.